MODULE 9

GitHub Repository Link: https://github.com/devcnx/csd440

Source Code

BrittaneyIndex.php

```
<?php
/**
 * Module 9.2 Programming Assignment — Index Page
 * CSD440 Server-Side Scripting
 *
 * Landing page for the Module 9 CRUD application. Displays a site map
 * with links to every PHP file in this assignment — the three new pages
 * (Index, Query, Add Record) plus the four carried over from Module 8
 * (Create Table, Populate Table, Query Table, Drop Table).
 *
 * Also attempts a lightweight database check so the user can see at a
 * glance whether the bperrymorgan_ollama_models table exists and how
 * many rows it holds before navigating deeper.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-10
 * @php     >= 8.0
 */

// — Configuration ————————————————————————————————
const DB_HOST = 'localhost';
const DB_USER = 'student1';
const DB_PASS = 'pass';
const DB_NAME = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
```

```
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Checks whether the Ollama models table currently exists.
 *
 * @param  mysqli $mysqli Active MySQLi connection.
 * @return bool         True if the table is present, false otherwise.
 */
function tableExists(mysqli $mysqli): bool
{
    $escaped = $mysqli->real_escape_string(DB_TABLE);
    $result = $mysqli->query("SHOW TABLES LIKE '" . $escaped . "'");
    if (!($result instanceof mysqli_result)) {
        return false;
    }
    $exists = $result->num_rows > 0;
    $result->free();
    return $exists;
}

/**
 * Returns the current row count of the Ollama models table.
 *
 * Returns 0 if the table does not exist or the query fails.
 *
 * @param  mysqli $mysqli Active MySQLi connection.
 * @return int          Number of rows in the table.
 */
function rowCount(mysqli $mysqli): int
{
    $result = $mysqli->query("SELECT COUNT(*) AS cnt FROM " . DB_TABLE);
    if (!($result instanceof mysqli_result)) {
        return 0;
    }
    $row = $result->fetch_assoc();
    $count = (int) ($row['cnt'] ?? 0);
    $result->free();
    return $count;
}

// ── Execute ──────────────────────────────────────────────────────
$connectError = null;
$tablePresent = false;
$rowCountVal = 0;

$mysqli = connectToDatabase($connectError);
if ($mysqli !== null) {
    $tablePresent = tableExists($mysqli);
    if ($tablePresent) {
        $rowCountVal = rowCount($mysqli);
```

```
    }
    $mysqli->close();
}

// Page metadata
$studentName = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 9.2 Programming Assignment — Index';
$courseName = 'CSD440 Server-Side Scripting';
$today = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Index — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
    <link rel="stylesheet" href="../../shared.css">
  </head>

  <body>
    <h1>Ollama Models Manager</h1>

    <div class="header-info">
      <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
      <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> —
        <?php echo htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
      <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
    </div>

    <?php if ($connectError !== null): ?>
      <div class="error-summary">
        <h2>Database Unavailable</h2>
        <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code>
database.</p>
        <ul class="error-list">
          <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
        </ul>
        <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
      </div>
    <?php else: ?>
      <div class="confirmation">
        <h2>Database Status</h2>
        <?php if ($tablePresent): ?>
          <p class="success-message">
            Table <code><?php echo DB_TABLE; ?></code> is present
            (<?php echo (int) $rowCountVal; ?> row<?php echo $rowCountVal !== 1 ? 's' : ''; ?>).
          </p>
        <?php else: ?>
          <p class="error-message">
            Table <code><?php echo DB_TABLE; ?></code> does not exist yet.
            Run <strong>Create Table</strong> and <strong>Populate Table</strong> first.
```

```
        </p>
      <?php endif; ?>
    </div>
<?php endif; ?>

<h2>Module 9 — New Pages</h2>
<table>
  <thead>
    <tr>
      <th>Page</th>
      <th>Description</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td><a href="BrittaneyIndex.php">Index</a></td>
      <td>This page — site map and database status check</td>
    </tr>
    <tr>
      <td><a href="BrittaneyQuery.php">Query</a></td>
      <td>Search models by name, quantization, installed status, or parameter range</td>
    </tr>
    <tr>
      <td><a href="BrittaneyForm.php">Add Record</a></td>
      <td>Insert a new Ollama model into the database via a validated form</td>
    </tr>
  </tbody>
</table>

<h2>Module 8 — Table Management</h2>
<table>
  <thead>
    <tr>
      <th>Page</th>
      <th>Description</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td><a href="BrittaneyCreateTable.php">Create Table</a></td>
      <td>Creates the <code><?php echo DB_TABLE; ?></code> table (idempotent)</td>
    </tr>
    <tr>
      <td><a href="BrittaneyPopulateTable.php">Populate Table</a></td>
      <td>Loads the 8-row seed dataset (INSERT … ON DUPLICATE KEY UPDATE)</td>
    </tr>
    <tr>
      <td><a href="BrittaneyQueryTable.php">Query Table</a></td>
      <td>Four predefined SELECT queries against the table</td>
    </tr>
    <tr>
      <td><a href="BrittaneyDropTable.php">Drop Table</a></td>
```

```
            <td>Removes the table for a clean reset (DROP TABLE IF EXISTS)</td>
          </tr>
        </tbody>
      </table>
    </body>

</html>
```

BrittaneyQuery.php

```
<?php
/**
 * Module 9.2 Programming Assignment — Query (Search) Page
 * CSD440 Server-Side Scripting
 *
 * Provides a search form that lets the user filter the bperrymorgan_ollama_models
 * table by model name (partial match), quantization scheme, installed status,
 * and parameter-count range. Uses prepared statements for all user-supplied
 * values to prevent SQL injection — consistent with the safe-DB-access pattern
 * established in Module 7 and carried through Module 8.
 *
 * The form submits via GET so search URLs are bookmarkable and shareable.
 * Results are rendered in a styled table with a count of matching rows.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-10
 * @php     >= 8.0
 */

// ── Configuration ──────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Builds and executes a parameterized SELECT query from the submitted
 * search criteria.
```

```
 *
 * All conditions are AND-ed together. The model_name filter uses
 * LIKE with wildcards for partial matching; all other filters are
 * exact-match or range-based. Parameters are bound via mysqli
 * prepared statements to prevent SQL injection.
 *
 * @param  mysqli     $mysqli      Active MySQLi connection.
 * @param  string     $modelName   Partial model name to search (empty = all).
 * @param  string     $quantization Quantization filter (empty = all).
 * @param  string     $installed   Installed filter: '1', '0', or '' (all).
 * @param  string     $paramMin    Minimum parameter count (empty = no lower bound).
 * @param  string     $paramMax    Maximum parameter count (empty = no upper bound).
 * @param  string|null &$error     Populated with the failure reason on error.
 * @return array                   Result rows as associative arrays.
 */
function searchModels(
    mysqli $mysqli,
    string $modelName,
    string $quantization,
    string $installed,
    string $paramMin,
    string $paramMax,
    ?string &$error = null
): array {
    $conditions = [];
    $types = '';
    $params = [];

    // Model name — partial match (LIKE)
    if ($modelName !== '') {
        $conditions[] = 'model_name LIKE ?';
        $types .= 's';
        $params[] = '%' . $modelName . '%';
    }

    // Quantization — exact match
    if ($quantization !== '') {
        $conditions[] = 'quantization = ?';
        $types .= 's';
        $params[] = $quantization;
    }

    // Installed status — exact match
    if ($installed === '1' || $installed === '0') {
        $conditions[] = 'is_installed = ?';
        $types .= 'i';
        $params[] = (int)$installed;
    }

    // Parameter count range
    if ($paramMin !== '') {
        $conditions[] = 'parameter_count >= ?';
```

```
        $types .= 'd';
        $params[] = (float)$paramMin;
    }
    if ($paramMax !== '') {
        $conditions[] = 'parameter_count <= ?';
        $types .= 'd';
        $params[] = (float)$paramMax;
    }

    // Build the full SQL statement
    $sql = "SELECT model_id, model_name, parameter_count, quantization, "
         . "size_gb, release_date, is_installed, use_case "
         . "FROM " . DB_TABLE;

    if (!empty($conditions)) {
        $sql .= ' WHERE ' . implode(' AND ', $conditions);
    }

    $sql .= ' ORDER BY parameter_count ASC, model_name ASC';

    $stmt = $mysqli->prepare($sql);
    if (!$stmt) {
        $error = $mysqli->error;
        return [];
    }

    // Bind parameters if any conditions were added
    if (!empty($params)) {
        $stmt->bind_param($types, ...$params);
    }

    if (!$stmt->execute()) {
        $error = $stmt->error;
        $stmt->close();
        return [];
    }

    $result = $stmt->get_result();
    $rows = [];
    while ($row = $result->fetch_assoc()) {
        $rows[] = $row;
    }
    $result->free();
    $stmt->close();

    return $rows;
}

// ── Process search ──────────────────────────────────────────
$modelName    = trim($_GET['model_name'] ?? '');
$quantization = trim($_GET['quantization'] ?? '');
```

```
$installed    = trim($_GET['installed'] ?? '');
$paramMin     = trim($_GET['param_min'] ?? '');
$paramMax     = trim($_GET['param_max'] ?? '');
$hasSearch    = ($modelName !== '' || $quantization !== '' || $installed !== ''
              || $paramMin !== '' || $paramMax !== '');

$connectError = null;
$searchError  = null;
$results      = [];

$mysqli = connectToDatabase($connectError);
if ($mysqli !== null) {
   if ($hasSearch) {
      $results = searchModels(
         $mysqli,
         $modelName,
         $quantization,
         $installed,
         $paramMin,
         $paramMax,
         $searchError
      );
   }
   $mysqli->close();
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 9.2 Programming Assignment — Query';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');

/**
 * Echoes a value or an em-dash when null/empty for cleaner table cells.
 *
 * @param  mixed $value Raw cell value from a result row.
 * @return string       HTML-safe display string.
 */
function cell(mixed $value): string
{
   if ($value === null || $value === '') {
      return '<span class="empty">&mdash;</span>';
   }
   return htmlspecialchars((string)$value, ENT_QUOTES, 'UTF-8');
}
?>
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Query — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
  <link rel="stylesheet" href="../../shared.css">
</head>

<body>
  <h1>Search Models</h1>

  <div class="header-info">
    <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
    <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
    <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
  </div>

<?php if ($connectError !== null): ?>
  <div class="error-summary">
    <h2>Database Unavailable</h2>
    <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code> database.</p>
    <ul class="error-list">
      <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
    </ul>
    <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
  </div>

<?php else: ?>

  <!-- ── Search Form ──────────────────────────────────────── -->
  <form method="GET" action="BrittaneyQuery.php" class="registration-form">
    <div class="form-group">
      <label for="model_name">Model Name</label>
      <input type="text" id="model_name" name="model_name"
          placeholder="Partial match (e.g., llama)"
          value="<?php echo htmlspecialchars($modelName, ENT_QUOTES, 'UTF-8'); ?>">
    </div>

    <div class="form-group">
      <label for="quantization">Quantization</label>
      <select id="quantization" name="quantization">
        <option value="">— Any —</option>
        <option value="Q4_K_M" <?php echo $quantization === 'Q4_K_M' ? 'selected' : ''; ?>>Q4_K_M</option>
        <option value="F16" <?php echo $quantization === 'F16' ? 'selected' : ''; ?>>F16</option>
      </select>
    </div>

    <div class="form-group">
      <label for="installed">Installed Locally</label>
      <select id="installed" name="installed">
        <option value="">— Any —</option>
        <option value="1" <?php echo $installed === '1' ? 'selected' : ''; ?>>Yes</option>
        <option value="0" <?php echo $installed === '0' ? 'selected' : ''; ?>>No</option>
      </select>
    </div>
```

```
    <div class="form-group">
      <label for="param_min">Parameter Count (Min, billions)</label>
      <input type="number" id="param_min" name="param_min" step="0.01" min="0"
          placeholder="e.g., 1"
          value="<?php echo htmlspecialchars($paramMin, ENT_QUOTES, 'UTF-8'); ?>">
    </div>

    <div class="form-group">
      <label for="param_max">Parameter Count (Max, billions)</label>
      <input type="number" id="param_max" name="param_max" step="0.01" min="0"
          placeholder="e.g., 100"
          value="<?php echo htmlspecialchars($paramMax, ENT_QUOTES, 'UTF-8'); ?>">
    </div>

    <div class="form-actions">
      <button type="submit">Search</button>
      <a href="BrittaneyQuery.php" class="btn-secondary">Reset</a>
    </div>
  </form>

  <!-- ── Search Results ──────────────────────────────────────── -->
<?php if ($hasSearch): ?>
  <h2>Search Results</h2>
  <div class="query-note">
    <p>Filters applied:
      <strong>Model name:</strong> <?php echo $modelName !== '' ? htmlspecialchars('"' . $modelName . '"',
ENT_QUOTES, 'UTF-8') : '<em>any</em>'; ?>,
      <strong>Quantization:</strong> <?php echo $quantization !== '' ? htmlspecialchars($quantization,
ENT_QUOTES, 'UTF-8') : '<em>any</em>'; ?>,
      <strong>Installed:</strong> <?php echo $installed === '1' ? 'Yes' : ($installed === '0' ? 'No' :
'<em>any</em>'); ?>,
      <strong>Params:</strong> <?php echo $paramMin !== '' ? htmlspecialchars($paramMin, ENT_QUOTES,
'UTF-8') . 'B' : '—'; ?>
      – <?php echo $paramMax !== '' ? htmlspecialchars($paramMax, ENT_QUOTES, 'UTF-8') . 'B' : '—'; ?>
    </p>
  </div>

<?php if ($searchError !== null): ?>
  <div class="error-summary">
    <p class="error-message">Query failed: <?php echo htmlspecialchars($searchError, ENT_QUOTES, 'UTF-8');
?></p>
  </div>
<?php elseif (empty($results)): ?>
  <p class="empty">No models matched your search criteria. Try broadening the filters.</p>
<?php else: ?>
  <table>
    <caption><?php echo count($results); ?> model<?php echo count($results) !== 1 ? 's' : ''; ?> found</caption>
    <thead>
      <tr>
        <th>ID</th>
        <th>Model</th>
```

```
            <th>Params (B)</th>
            <th>Quant</th>
            <th>Size (GB)</th>
            <th>Released</th>
            <th>Installed</th>
            <th>Use Case</th>
          </tr>
        </thead>
        <tbody>
<?php foreach ($results as $row): ?>
          <tr>
            <td><?php echo (int)$row['model_id']; ?></td>
            <td><code><?php echo cell($row['model_name']); ?></code></td>
            <td><?php echo cell($row['parameter_count']); ?></td>
            <td><?php echo cell($row['quantization']); ?></td>
            <td><?php echo cell($row['size_gb']); ?></td>
            <td><?php echo cell($row['release_date']); ?></td>
            <td>
<?php if ((int)$row['is_installed'] === 1): ?>
                <span class="flag-true">Yes</span>
<?php else: ?>
                <span class="flag-false">No</span>
<?php endif; ?>
            </td>
            <td><?php echo cell($row['use_case']); ?></td>
          </tr>
<?php endforeach; ?>
        </tbody>
      </table>
<?php endif; ?>
<?php else: ?>
      <div class="query-note">
        <p>Enter search criteria above and click <strong>Search</strong> to filter models.
          Leave all fields blank to list every row.</p>
      </div>
<?php endif; ?>

<?php endif; /* connection ok */ ?>

      <div class="form-actions">
        <a href="BrittaneyIndex.php">Index</a>
        <a href="BrittaneyForm.php" class="btn-secondary">Add Record</a>
      </div>
</body>

</html>
```

BrittaneyForm.php

```
<?php
/**
 * Module 9.2 Programming Assignment — Add Record Form
 * CSD440 Server-Side Scripting
 *
 * Presents a form for inserting a new row into the bperrymorgan_ollama_models
 * table. Validates all fields server-side before executing a prepared INSERT
 * statement. On success, displays a confirmation page with the inserted data.
 * On validation failure, repopulates the form with the submitted values and
 * lists the errors that need correction.
 *
 * Validation rules:
 *   - model_name:    required, max 100 chars, must be unique in the table
 *   - parameter_count: required, numeric, >= 0.01
 *   - quantization:   required, max 20 chars
 *   - size_gb:        required, numeric, >= 0.01
 *   - release_date:    required, valid date (YYYY-MM-DD)
 *   - is_installed:    required, must be '0' or '1'
 *   - use_case:       required, max 255 chars
 *
 * Uses session-based PRG (Post/Redirect/Get) so refreshed submissions
 * do not create duplicate rows.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-10
 * @php     >= 8.0
 */

session_start();

// ── Configuration ──────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
```

```
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Checks whether a model name already exists in the table.
 *
 * Used during validation to enforce the UNIQUE constraint on
 * model_name before attempting the INSERT.
 *
 * @param  mysqli $mysqli    Active MySQLi connection.
 * @param  string $modelName The model name to check.
 * @return bool              True if the name already exists.
 */
function modelNameExists(mysqli $mysqli, string $modelName): bool
{
    $stmt = $mysqli->prepare("SELECT COUNT(*) FROM " . DB_TABLE . " WHERE model_name = ?");
    if (!$stmt) {
        return false;
    }
    $stmt->bind_param('s', $modelName);
    $stmt->execute();
    $stmt->bind_result($count);
    $stmt->fetch();
    $stmt->close();
    return $count > 0;
}

/**
 * Inserts a new model record using a prepared statement.
 *
 * @param  mysqli      $mysqli     Active MySQLi connection.
 * @param  array       $data       Validated and sanitized data.
 * @param  string|null &$error     Populated on failure.
 * @return bool                    True on success, false on failure.
 */
function insertModel(mysqli $mysqli, array $data, ?string &$error = null): bool
{
    $sql = "INSERT INTO " . DB_TABLE . "
            (model_name, parameter_count, quantization, size_gb, release_date, is_installed, use_case)
            VALUES (?, ?, ?, ?, ?, ?, ?)";

    $stmt = $mysqli->prepare($sql);
    if (!$stmt) {
        $error = $mysqli->error;
        return false;
    }
```

```
    $stmt->bind_param(
        'sdsdsis',
        $data['model_name'],
        $data['parameter_count'],
        $data['quantization'],
        $data['size_gb'],
        $data['release_date'],
        $data['is_installed'],
        $data['use_case']
    );

    if (!$stmt->execute()) {
        $error = $stmt->error;
        $stmt->close();
        return false;
    }

    $stmt->close();
    return true;
}

/**
 * Validates the submitted form data and returns an array of error messages.
 *
 * Each key matches a form field name so errors can be displayed inline.
 * Returns an empty array when all fields pass validation.
 *
 * @param  array $postData The raw POST data.
 * @param  bool  $checkUnique If true, checks model_name uniqueness against DB.
 * @return array Associative array of [field => error message].
 */
function validateFormData(array $postData, bool $checkUnique = false): array
{
    $errors = [];

    // Model name — required, max 100 chars, must be unique
    $modelName = trim($postData['model_name'] ?? '');
    if ($modelName === '') {
        $errors['model_name'] = 'Model name is required.';
    } elseif (strlen($modelName) > 100) {
        $errors['model_name'] = 'Model name must be 100 characters or fewer.';
    }

    // Parameter count — required, must be numeric and positive
    $paramCount = trim($postData['parameter_count'] ?? '');
    if ($paramCount === '') {
        $errors['parameter_count'] = 'Parameter count is required.';
    } elseif (!is_numeric($paramCount) || (float)$paramCount < 0.01) {
        $errors['parameter_count'] = 'Parameter count must be a positive number.';
    }

    // Quantization — required, max 20 chars
```

```
$quantization = trim($postData['quantization'] ?? '');
if ($quantization === '') {
   $errors['quantization'] = 'Quantization is required.';
} elseif (strlen($quantization) > 20) {
   $errors['quantization'] = 'Quantization must be 20 characters or fewer.';
}

// Size in GB — required, must be numeric and positive
$sizeGb = trim($postData['size_gb'] ?? '');
if ($sizeGb === '') {
   $errors['size_gb'] = 'Size (GB) is required.';
} elseif (!is_numeric($sizeGb) || (float)$sizeGb < 0.01) {
   $errors['size_gb'] = 'Size must be a positive number.';
}

// Release date — required, must be a valid date
$releaseDate = trim($postData['release_date'] ?? '');
if ($releaseDate === '') {
   $errors['release_date'] = 'Release date is required.';
} else {
   $d = date_create($releaseDate);
   if (!$d || $d->format('Y-m-d') !== $releaseDate) {
      $errors['release_date'] = 'Release date must be a valid date (YYYY-MM-DD).';
   }
}

// Installed status — required, must be 0 or 1
$installed = trim($postData['is_installed'] ?? '');
if ($installed !== '0' && $installed !== '1') {
   $errors['is_installed'] = 'Please select whether the model is installed.';
}

// Use case — required, max 255 chars
$useCase = trim($postData['use_case'] ?? '');
if ($useCase === '') {
   $errors['use_case'] = 'Use case is required.';
} elseif (strlen($useCase) > 255) {
   $errors['use_case'] = 'Use case must be 255 characters or fewer.';
}

// Unique check for model_name — only if no earlier errors on that field
if ($checkUnique && !isset($errors['model_name']) && $modelName !== '') {
   $connErr = null;
   $mysqli = connectToDatabase($connErr);
   if ($mysqli !== null) {
      if (modelNameExists($mysqli, $modelName)) {
         $errors['model_name'] = 'A model with this name already exists.';
      }
      $mysqli->close();
   }
}
```

```
    return $errors;
}

/**
 * Sanitizes validated POST data for safe display.
 *
 * @param  array $postData The raw POST data.
 * @return array Sanitized data safe for HTML output.
 */
function sanitizeFormData(array $postData): array
{
    return [
        'model_name'      => htmlspecialchars(trim($postData['model_name'] ?? ''), ENT_QUOTES, 'UTF-8'),
        'parameter_count' => htmlspecialchars(trim($postData['parameter_count'] ?? ''), ENT_QUOTES, 'UTF-8'),
        'quantization'    => htmlspecialchars(trim($postData['quantization'] ?? ''), ENT_QUOTES, 'UTF-8'),
        'size_gb'         => htmlspecialchars(trim($postData['size_gb'] ?? ''), ENT_QUOTES, 'UTF-8'),
        'release_date'    => htmlspecialchars(trim($postData['release_date'] ?? ''), ENT_QUOTES, 'UTF-8'),
        'is_installed'    => trim($postData['is_installed'] ?? ''),
        'use_case'        => htmlspecialchars(trim($postData['use_case'] ?? ''), ENT_QUOTES, 'UTF-8'),
    ];
}

// ── Handle POST submission (PRG pattern) ──────────────────────
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $errors = validateFormData($_POST, true);

    if (empty($errors)) {
        // All valid — insert the record
        $connErr = null;
        $mysqli = connectToDatabase($connErr);

        if ($mysqli === null) {
            // Store error in session and redirect back
            $_SESSION['form_errors'] = ['database' => 'Could not connect to the database: ' . $connErr];
            $_SESSION['form_data'] = $_POST;
            header('Location: BrittaneyForm.php');
            exit;
        }

        $insertError = null;
        $success = insertModel($mysqli, [
            'model_name'      => trim($_POST['model_name']),
            'parameter_count' => (float)trim($_POST['parameter_count']),
            'quantization'    => trim($_POST['quantization']),
            'size_gb'         => (float)trim($_POST['size_gb']),
            'release_date'    => trim($_POST['release_date']),
            'is_installed'    => (int)trim($_POST['is_installed']),
            'use_case'        => trim($_POST['use_case']),
        ], $insertError);

        $mysqli->close();
```

```
        if ($success) {
            $_SESSION['insert_success'] = true;
            $_SESSION['inserted_data'] = sanitizeFormData($_POST);
            header('Location: BrittaneyForm.php');
            exit;
        }

        // Insert failed — redirect back with error
        $_SESSION['form_errors'] = ['database' => $insertError ?? 'Failed to insert the record.'];
        $_SESSION['form_data'] = $_POST;
        header('Location: BrittaneyForm.php');
        exit;
    }

    // Validation failed — store errors and data, redirect back
    $_SESSION['form_errors'] = $errors;
    $_SESSION['form_data'] = $_POST;
    header('Location: BrittaneyForm.php');
    exit;
}

// ── GET request: display form ──────────────────────────────
$errors     = $_SESSION['form_errors'] ?? [];
$oldData    = $_SESSION['form_data'] ?? [];
$successMsg = $_SESSION['insert_success'] ?? false;
$inserted   = $_SESSION['inserted_data'] ?? [];
unset($_SESSION['form_errors'], $_SESSION['form_data'], $_SESSION['insert_success'],
$_SESSION['inserted_data']);

/**
 * Returns old form data for a field, HTML-escaped.
 *
 * @param  string $field    The field name.
 * @param  array  $oldData  The stored old data.
 * @return string           Escaped value or empty string.
 */
function old(string $field, array $oldData): string
{
    return isset($oldData[$field]) ? htmlspecialchars(trim($oldData[$field]), ENT_QUOTES, 'UTF-8') : '';
}

/**
 * Returns 'selected' if a dropdown option matches the old value.
 *
 * @param  string $field    The field name.
 * @param  string $value    The option value to check.
 * @param  array  $oldData  The stored old data.
 * @return string           'selected' or empty string.
 */
function selected(string $field, string $value, array $oldData): string
{
    return (isset($oldData[$field]) && trim($oldData[$field]) === $value) ? ' selected' : '';
```

```
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 9.2 Programming Assignment — Add Record';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Add Record — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
  <link rel="stylesheet" href="../../shared.css">
</head>

<body>
  <h1>Add a Model Record</h1>

  <div class="header-info">
    <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
    <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
    <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
  </div>

<?php if ($successMsg && !empty($inserted)): ?>
  <!-- ── Success Confirmation ──────────────────────────────── -->
  <div class="confirmation">
    <h2>Record Added Successfully</h2>
    <p class="success-message">The model has been inserted into the <code><?php echo DB_TABLE; ?></code>
table.</p>

    <table class="data-display">
      <caption>Inserted Record</caption>
      <tbody>
        <tr>
          <td>Model Name</td>
          <td><code><?php echo $inserted['model_name']; ?></code></td>
        </tr>
        <tr>
          <td>Parameter Count</td>
          <td><?php echo $inserted['parameter_count']; ?> B</td>
        </tr>
        <tr>
          <td>Quantization</td>
          <td><?php echo $inserted['quantization']; ?></td>
        </tr>
        <tr>
          <td>Size (GB)</td>
```

```
            <td><?php echo $inserted['size_gb']; ?> GB</td>
          </tr>
          <tr>
            <td>Release Date</td>
            <td><?php echo $inserted['release_date']; ?></td>
          </tr>
          <tr>
            <td>Installed Locally</td>
            <td><?php echo $inserted['is_installed'] === '1' ? '<span class="flag-true">Yes</span>' : '<span
class="flag-false">No</span>'; ?></td>
          </tr>
          <tr>
            <td>Use Case</td>
            <td><?php echo $inserted['use_case']; ?></td>
          </tr>
        </tbody>
      </table>
    </div>

    <div class="form-actions">
      <a href="BrittaneyForm.php">Add Another Record</a>
      <a href="BrittaneyQuery.php" class="btn-secondary">Search Models</a>
      <a href="BrittaneyIndex.php" class="btn-secondary">Index</a>
    </div>

<?php else: ?>
    <!-- ── Error Messages ──────────────────────────────────────── -->
<?php if (!empty($errors)): ?>
    <div class="error-summary">
      <h2>Correction Required</h2>
      <p class="error-message">Please fix the following issues and resubmit:</p>
      <ul class="error-list">
<?php foreach ($errors as $message): ?>
        <li><?php echo htmlspecialchars($message, ENT_QUOTES, 'UTF-8'); ?></li>
<?php endforeach; ?>
      </ul>
    </div>
<?php endif; ?>

    <!-- ── Add Record Form ─────────────────────────────────────── -->
    <form method="POST" action="BrittaneyForm.php" class="registration-form">
      <div class="form-group">
        <label for="model_name">Model Name <span class="required">*</span></label>
        <input type="text" id="model_name" name="model_name"
            placeholder="e.g., llama3.1:8b" maxlength="100"
            value="<?php echo old('model_name', $oldData); ?>" required>
<?php if (isset($errors['model_name'])): ?>
        <span class="error"><?php echo htmlspecialchars($errors['model_name'], ENT_QUOTES, 'UTF-8');
?></span>
<?php endif; ?>
      </div>
```

```
    <div class="form-group">
      <label for="parameter_count">Parameter Count (Billions) <span class="required">*</span></label>
      <input type="number" id="parameter_count" name="parameter_count"
          step="0.01" min="0.01" placeholder="e.g., 8.00"
          value="<?php echo old('parameter_count', $oldData); ?>" required>
<?php if (isset($errors['parameter_count'])): ?>
      <span class="error"><?php echo htmlspecialchars($errors['parameter_count'], ENT_QUOTES, 'UTF-8');
?></span>
<?php endif; ?>
    </div>

    <div class="form-group">
      <label for="quantization">Quantization <span class="required">*</span></label>
      <input type="text" id="quantization" name="quantization"
          placeholder="e.g., Q4_K_M" maxlength="20"
          value="<?php echo old('quantization', $oldData); ?>" required>
<?php if (isset($errors['quantization'])): ?>
      <span class="error"><?php echo htmlspecialchars($errors['quantization'], ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
    </div>

    <div class="form-group">
      <label for="size_gb">Size (GB) <span class="required">*</span></label>
      <input type="number" id="size_gb" name="size_gb"
          step="0.01" min="0.01" placeholder="e.g., 4.92"
          value="<?php echo old('size_gb', $oldData); ?>" required>
<?php if (isset($errors['size_gb'])): ?>
      <span class="error"><?php echo htmlspecialchars($errors['size_gb'], ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
    </div>

    <div class="form-group">
      <label for="release_date">Release Date <span class="required">*</span></label>
      <input type="date" id="release_date" name="release_date"
          value="<?php echo old('release_date', $oldData); ?>" required>
<?php if (isset($errors['release_date'])): ?>
      <span class="error"><?php echo htmlspecialchars($errors['release_date'], ENT_QUOTES, 'UTF-8');
?></span>
<?php endif; ?>
    </div>

    <div class="form-group">
      <label for="is_installed">Installed Locally <span class="required">*</span></label>
      <select id="is_installed" name="is_installed" required>
        <option value="">— Select —</option>
        <option value="1"<?php echo selected('is_installed', '1', $oldData); ?>>Yes</option>
        <option value="0"<?php echo selected('is_installed', '0', $oldData); ?>>No</option>
      </select>
<?php if (isset($errors['is_installed'])): ?>
      <span class="error"><?php echo htmlspecialchars($errors['is_installed'], ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
    </div>
```

```
      <div class="form-group">
        <label for="use_case">Use Case <span class="required">*</span></label>
        <input type="text" id="use_case" name="use_case"
             placeholder="e.g., General-purpose chat and coding assistance" maxlength="255"
             value="<?php echo old('use_case', $oldData); ?>" required>
<?php if (isset($errors['use_case'])): ?>
        <span class="error"><?php echo htmlspecialchars($errors['use_case'], ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
      </div>

      <div class="form-actions">
        <button type="submit">Add Record</button>
        <a href="BrittaneyForm.php" class="btn-secondary">Reset</a>
      </div>

      <p class="form-note"><span class="required">*</span> Required fields</p>
    </form>
<?php endif; ?>

    <div class="form-actions" style="margin-top: 30px;">
      <a href="BrittaneyIndex.php">Index</a>
      <a href="BrittaneyQuery.php" class="btn-secondary">Search Models</a>
    </div>
</body>

</html>
```

BrittaneyCreateTable.php

```
<?php
/**
 * Module 9.2 — Create Table (carried from Module 8.2)
 * CSD440 Server-Side Scripting
 *
 * Creates the bperrymorgan_ollama_models table inside the course-supplied
 * baseball_01 database using MySQLi. The table catalogs locally available
 * Ollama language models — chosen as a topic because it ties into ongoing
 * AI-enablement work and naturally exercises multiple data types.
 *
 * Schema (8 columns, 5 distinct data types):
 *   - model_id       INT          AUTO_INCREMENT PRIMARY KEY
 *   - model_name     VARCHAR(100) UNIQUE
 *   - parameter_count DECIMAL(6,2) — model size in billions of parameters
 *   - quantization   VARCHAR(20)  — e.g., Q4_K_M, F16
 *   - size_gb        DECIMAL(6,2) — disk footprint in gigabytes
 *   - release_date   DATE
 *   - is_installed   TINYINT(1)   — boolean flag, 1 = pulled locally
 *   - use_case       VARCHAR(255)
 *
 * Intended run order: Create → Populate → Query. Drop is available for
 * cleanup between test runs.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-10
 * @php     >= 8.0
 */

// ── Configuration ──────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
```

```
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Issues the CREATE TABLE statement against the active connection.
 *
 * Uses CREATE TABLE IF NOT EXISTS so the script is safely idempotent —
 * re-running it does not error out when the table already exists.
 *
 * @param  mysqli      $mysqli Active MySQLi connection.
 * @param  string|null &$error Populated with the failure reason on error.
 * @return bool                True if the table now exists, false on error.
 */
function createOllamaModelsTable(mysqli $mysqli, ?string &$error = null): bool
{
    $sql = "CREATE TABLE IF NOT EXISTS " . DB_TABLE . " (
        model_id       INT          NOT NULL AUTO_INCREMENT,
        model_name     VARCHAR(100) NOT NULL UNIQUE,
        parameter_count DECIMAL(6,2) NOT NULL,
        quantization   VARCHAR(20)  NOT NULL,
        size_gb        DECIMAL(6,2) NOT NULL,
        release_date   DATE         NOT NULL,
        is_installed   TINYINT(1)   NOT NULL DEFAULT 0,
        use_case       VARCHAR(255) NOT NULL,
        PRIMARY KEY (model_id)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4";

    if (!$mysqli->query($sql)) {
        $error = $mysqli->error;
        return false;
    }
    return true;
}

/**
 * Reads the live table schema via DESCRIBE for display confirmation.
 *
 * Returns the column descriptors as an array of associative rows. An
 * empty array indicates the DESCRIBE query failed or the table is missing.
 *
 * @param  mysqli $mysqli Active MySQLi connection.
 * @return array          List of column descriptors (Field, Type, Null, …).
 */
function describeTable(mysqli $mysqli): array
{
    $rows = [];
    $result = $mysqli->query("DESCRIBE " . DB_TABLE);
    if ($result instanceof mysqli_result) {
        while ($row = $result->fetch_assoc()) {
```

```
            $rows[] = $row;
        }
        $result->free();
    }
    return $rows;
}

// — Execute ————————————————————————————————————————
$connectError = null;
$createError  = null;
$schemaRows   = [];
$tableCreated = false;

$mysqli = connectToDatabase($connectError);
if ($mysqli !== null) {
    $tableCreated = createOllamaModelsTable($mysqli, $createError);
    if ($tableCreated) {
        $schemaRows = describeTable($mysqli);
    }
    $mysqli->close();
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 9.2 — Create Table';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Create Table — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
    <link rel="stylesheet" href="../../shared.css">
</head>

<body>
    <h1>Create Table</h1>

    <div class="header-info">
        <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
        <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
        <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
    </div>

<?php if ($connectError !== null): ?>
    <div class="error-summary">
        <h2>Connection Failed</h2>
        <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code> database.</p>
```

```
      <ul class="error-list">
        <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
      </ul>
      <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
    </div>

<?php elseif (!$tableCreated): ?>
    <div class="error-summary">
      <h2>Create Failed</h2>
      <p class="error-message">The <code><?php echo DB_TABLE; ?></code> table could not be created.</p>
      <ul class="error-list">
        <li><?php echo htmlspecialchars($createError ?? 'Unknown error.', ENT_QUOTES, 'UTF-8'); ?></li>
      </ul>
    </div>

<?php else: ?>
    <div class="confirmation">
      <h2>Table Ready</h2>
      <p class="success-message">
        The <code><?php echo DB_TABLE; ?></code> table is in place
        (created if missing) inside the <code><?php echo DB_NAME; ?></code> database.
      </p>
      <div class="query-note">
        <strong>SQL executed:</strong>
        <code>CREATE TABLE IF NOT EXISTS <?php echo DB_TABLE; ?> ( … 8 columns … )</code>
      </div>
      <p>Next step: populate the table by running
        <code><a href="BrittaneyPopulateTable.php">BrittaneyPopulateTable.php</a></code>.</p>
    </div>

    <h2>Confirmed Schema</h2>
    <table>
      <caption>DESCRIBE <?php echo DB_TABLE; ?></caption>
      <thead>
        <tr>
          <th>Field</th>
          <th>Type</th>
          <th>Null</th>
          <th>Key</th>
          <th>Default</th>
          <th>Extra</th>
        </tr>
      </thead>
      <tbody>
<?php foreach ($schemaRows as $row): ?>
        <tr>
          <td><code><?php echo htmlspecialchars($row['Field'], ENT_QUOTES, 'UTF-8'); ?></code></td>
          <td><?php echo htmlspecialchars($row['Type'], ENT_QUOTES, 'UTF-8'); ?></td>
          <td><?php echo htmlspecialchars($row['Null'], ENT_QUOTES, 'UTF-8'); ?></td>
          <td><?php echo htmlspecialchars($row['Key'], ENT_QUOTES, 'UTF-8'); ?></td>
          <td><?php echo htmlspecialchars((string)($row['Default'] ?? ''), ENT_QUOTES, 'UTF-8'); ?></td>
          <td><?php echo htmlspecialchars($row['Extra'], ENT_QUOTES, 'UTF-8'); ?></td>
```

```
        </tr>
<?php endforeach; ?>
      </tbody>
   </table>
<?php endif; ?>

   <div class="form-actions">
      <a href="BrittaneyIndex.php">Index</a>
      <a href="BrittaneyPopulateTable.php">Populate Table</a>
      <a href="BrittaneyQueryTable.php" class="btn-secondary">Query Table</a>
      <a href="BrittaneyDropTable.php" class="btn-secondary">Drop Table</a>
   </div>
</body>

</html>
```

BrittaneyPopulateTable.php

```
<?php
/**
 * Module 9.2 — Populate Table (carried from Module 8.2)
 * CSD440 Server-Side Scripting
 *
 * Inserts seed data into the bperrymorgan_ollama_models table inside the
 * course-supplied baseball_01 database. Uses a parameterized prepared
 * statement (mysqli::prepare + bind_param) so the load step also serves
 * as a working demonstration of the safe-DB-access pattern from M7.1.
 *
 * The seed dataset describes eight Ollama models that span a realistic
 * range of parameter sizes, quantizations, release dates, and on-disk
 * footprints — chosen so the M9 query work has interesting data to filter
 * and aggregate over.
 *
 * Idempotency: the INSERT uses ON DUPLICATE KEY UPDATE keyed on
 * model_name so re-running this script refreshes existing rows rather
 * than throwing duplicate-key errors. The summary distinguishes inserts
 * from updates so the result is unambiguous.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-10
 * @php     >= 8.0
 */

// ── Configuration ──────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

// Seed rows, ordered by release date.
// Each row: [model_name, parameter_count_billions, quantization,
//            size_gb, release_date, is_installed, use_case]
const SEED_ROWS = [
    ['llava:13b',              13.00, 'Q4_K_M', 8.00, '2024-01-30', 0, 'Vision-language model for image understanding'],
    ['nomic-embed-text:latest',  0.14, 'F16',    0.27, '2024-02-01', 1, 'Text embeddings for RAG pipelines'],
    ['phi3:mini',               3.80, 'Q4_K_M', 2.30, '2024-04-23', 1, 'Lightweight reasoning on limited hardware'],
    ['gemma2:9b',               9.00, 'Q4_K_M', 5.44, '2024-06-27', 1, 'Open Google model, balanced performance'],
    ['mistral-nemo:12b',       12.00, 'Q4_K_M', 7.07, '2024-07-18', 1, 'Multilingual chat with long context window'],
    ['llama3.1:8b',             8.00, 'Q4_K_M', 4.92, '2024-07-23', 1, 'General-purpose chat and coding assistance'],
    ['llama3.1:70b',           70.00, 'Q4_K_M',39.50, '2024-07-23', 0, 'High-quality reasoning, slower local inference'],
    ['qwen2.5-coder:7b',        7.00, 'Q4_K_M', 4.36, '2024-09-19', 1, 'Code generation and refactoring'],
];

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
```

```
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Inserts the seed dataset into the table using a prepared statement.
 *
 * Loops the SEED_ROWS constant, binding each row's values into the same
 * prepared INSERT to avoid SQL injection and re-parsing overhead. The
 * ON DUPLICATE KEY UPDATE clause keys on model_name (the UNIQUE column),
 * so re-running the script refreshes rows rather than failing.
 *
 * affected_rows semantics under ON DUPLICATE KEY UPDATE:
 *   1 = new row inserted
 *   2 = existing row updated
 *   0 = existing row matched but no values changed (counted as update)
 *
 * @param  mysqli      $mysqli Active MySQLi connection.
 * @param  string|null &$error Populated with the failure reason on error.
 * @return array               Counts: ['inserted' => int, 'updated' => int,
 *                             'total' => int, 'failed' => int].
 */
function populateOllamaModelsTable(mysqli $mysqli, ?string &$error = null): array
{
    $sql = "INSERT INTO " . DB_TABLE . "
        (model_name, parameter_count, quantization, size_gb, release_date, is_installed, use_case)
        VALUES (?, ?, ?, ?, ?, ?, ?)
        ON DUPLICATE KEY UPDATE
            parameter_count = VALUES(parameter_count),
            quantization    = VALUES(quantization),
            size_gb         = VALUES(size_gb),
            release_date    = VALUES(release_date),
            is_installed    = VALUES(is_installed),
            use_case        = VALUES(use_case)";

    $stmt = $mysqli->prepare($sql);
    if (!$stmt) {
        $error = $mysqli->error;
        return ['inserted' => 0, 'updated' => 0, 'total' => count(SEED_ROWS), 'failed' => count(SEED_ROWS)];
```

```
    }

    $inserted = 0;
    $updated  = 0;
    $failed   = 0;

    foreach (SEED_ROWS as $row) {
        // Type string: s = string, d = decimal/double, i = integer
        // Order matches the VALUES (?, ?, ?, ?, ?, ?, ?) above.
        $name       = $row[0];
        $params     = $row[1];
        $quant      = $row[2];
        $sizeGb     = $row[3];
        $releaseDate = $row[4];
        $installed  = $row[5];
        $useCase    = $row[6];

        $stmt->bind_param('sdsdsis', $name, $params, $quant, $sizeGb, $releaseDate, $installed, $useCase);

        if (!$stmt->execute()) {
            $failed++;
            continue;
        }

        // affected_rows: 1 = insert, 2 = update via ON DUPLICATE KEY,
        // 0 = matched but unchanged. Treat 0 and 2 both as "already there."
        if ($stmt->affected_rows === 1) {
            $inserted++;
        } else {
            $updated++;
        }
    }
    $stmt->close();

    return [
        'inserted' => $inserted,
        'updated'  => $updated,
        'total'    => count(SEED_ROWS),
        'failed'   => $failed,
    ];
}

/**
 * Reads every row currently in the table for post-load verification.
 *
 * @param  mysqli $mysqli Active MySQLi connection.
 * @return array          List of row associative arrays, ordered by model_id.
 */
function fetchAllRows(mysqli $mysqli): array
{
    $rows = [];
    $result = $mysqli->query("SELECT * FROM " . DB_TABLE . " ORDER BY model_id");
```

```
    if ($result instanceof mysqli_result) {
        while ($row = $result->fetch_assoc()) {
            $rows[] = $row;
        }
        $result->free();
    }
    return $rows;
}

// ── Execute ──────────────────────────────────────────────────────
$connectError  = null;
$populateError = null;
$counts       = ['inserted' => 0, 'updated' => 0, 'total' => 0, 'failed' => 0];
$rows         = [];
$populated    = false;

$mysqli = connectToDatabase($connectError);
if ($mysqli !== null) {
    $counts    = populateOllamaModelsTable($mysqli, $populateError);
    $populated = $populateError === null && $counts['failed'] === 0;
    if ($populateError === null) {
        $rows = fetchAllRows($mysqli);
    }
    $mysqli->close();
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 9.2 — Populate Table';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Populate Table — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
    <link rel="stylesheet" href="../../shared.css">
</head>

<body>
    <h1>Populate Table</h1>

    <div class="header-info">
        <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
        <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
        <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
    </div>
```

```
<?php if ($connectError !== null): ?>
  <div class="error-summary">
    <h2>Connection Failed</h2>
    <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code> database.</p>
    <ul class="error-list">
      <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
    </ul>
    <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
  </div>

<?php elseif ($populateError !== null): ?>
  <div class="error-summary">
    <h2>Populate Failed</h2>
    <p class="error-message">The prepared INSERT could not be initialized.</p>
    <ul class="error-list">
      <li><?php echo htmlspecialchars($populateError, ENT_QUOTES, 'UTF-8'); ?></li>
    </ul>
    <p>If the error mentions a missing table, run
      <code><a href="BrittaneyCreateTable.php">BrittaneyCreateTable.php</a></code> first.</p>
  </div>

<?php else: ?>
  <div class="confirmation">
    <h2>Seed Load Complete</h2>
    <p class="success-message">
      Processed <?php echo (int)$counts['total']; ?> rows against
      <code><?php echo DB_TABLE; ?></code>:
      <strong><?php echo (int)$counts['inserted']; ?></strong> inserted,
      <strong><?php echo (int)$counts['updated']; ?></strong> updated.
<?php if ($counts['failed'] > 0): ?>
      <span class="error"><?php echo (int)$counts['failed']; ?> failed.</span>
<?php endif; ?>
    </p>
    <div class="query-note">
      <strong>SQL pattern:</strong>
      <code>INSERT INTO <?php echo DB_TABLE; ?> ( … ) VALUES (?, ?, ?, ?, ?, ?, ?) ON DUPLICATE KEY UPDATE
…</code>
      — bound seven times per row using <code>mysqli::prepare</code> + <code>bind_param</code>.
    </div>
    <p>Next step: verify with
      <code><a href="BrittaneyQueryTable.php">BrittaneyQueryTable.php</a></code>.</p>
  </div>

  <h2>Table Contents After Load</h2>
  <table>
    <caption>SELECT * FROM <?php echo DB_TABLE; ?> ORDER BY model_id</caption>
    <thead>
      <tr>
        <th>ID</th>
        <th>Model</th>
        <th>Params (B)</th>
        <th>Quant</th>
```

```
            <th>Size (GB)</th>
            <th>Released</th>
            <th>Installed</th>
            <th>Use Case</th>
          </tr>
        </thead>
        <tbody>
<?php if (empty($rows)): ?>
          <tr><td colspan="8" class="empty">No rows present.</td></tr>
<?php else: foreach ($rows as $row): ?>
          <tr>
            <td><?php echo (int)$row['model_id']; ?></td>
            <td><code><?php echo htmlspecialchars($row['model_name'], ENT_QUOTES, 'UTF-8'); ?></code></td>
            <td><?php echo htmlspecialchars($row['parameter_count'], ENT_QUOTES, 'UTF-8'); ?></td>
            <td><?php echo htmlspecialchars($row['quantization'], ENT_QUOTES, 'UTF-8'); ?></td>
            <td><?php echo htmlspecialchars($row['size_gb'], ENT_QUOTES, 'UTF-8'); ?></td>
            <td><?php echo htmlspecialchars($row['release_date'], ENT_QUOTES, 'UTF-8'); ?></td>
            <td>
<?php if ((int)$row['is_installed'] === 1): ?>
                <span class="flag-true">Yes</span>
<?php else: ?>
                <span class="flag-false">No</span>
<?php endif; ?>
            </td>
            <td><?php echo htmlspecialchars($row['use_case'], ENT_QUOTES, 'UTF-8'); ?></td>
          </tr>
<?php endforeach; endif; ?>
        </tbody>
      </table>
<?php endif; ?>

      <div class="form-actions">
        <a href="BrittaneyIndex.php">Index</a>
        <a href="BrittaneyQueryTable.php">Query Table</a>
        <a href="BrittaneyCreateTable.php" class="btn-secondary">Create Table</a>
        <a href="BrittaneyDropTable.php" class="btn-secondary">Drop Table</a>
      </div>
    </body>

    </html>
```

BrittaneyQueryTable.php

```
<?php
/**
 * Module 9.2 — Query Table (carried from Module 8.2)
 * CSD440 Server-Side Scripting
 *
 * Runs four SELECT queries against the bperrymorgan_ollama_models table
 * inside the course-supplied baseball_01 database to demonstrate the
 * table is populated and behaves as expected. Each query exercises a
 * different SQL feature so the demo also covers grading rubric breadth:
 *
 *   1. Full-table read with explicit column ordering.
 *   2. Filtered read (WHERE + ORDER BY) — locally installed models only.
 *   3. Aggregate read (COUNT, SUM, AVG, ROUND) — fleet summary.
 *   4. Grouped read (GROUP BY quantization) — counts per quant scheme.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-10
 * @php     >= 8.0
 */

// ── Configuration ──────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Runs an arbitrary SELECT and returns the result rows.
```

```
 *
 * Wraps the raw mysqli::query result so the caller works only with plain
 * arrays. Errors are surfaced via the $error reference instead of being
 * thrown so the calling page can render multiple queries independently
 * (one failure should not abort the rest of the page).
 *
 * @param  mysqli      $mysqli Active MySQLi connection.
 * @param  string      $sql    The SQL SELECT statement to execute.
 * @param  string|null &$error Populated with the failure reason on error.
 * @return array               Result rows; empty array on error or no rows.
 */
function runSelect(mysqli $mysqli, string $sql, ?string &$error = null): array
{
    $rows = [];
    $result = $mysqli->query($sql);
    if (!($result instanceof mysqli_result)) {
        $error = $mysqli->error;
        return [];
    }
    while ($row = $result->fetch_assoc()) {
        $rows[] = $row;
    }
    $result->free();
    return $rows;
}

// ── Execute ──────────────────────────────────────────────────────────
$connectError = null;
$mysqli       = connectToDatabase($connectError);

// Each query gets its own error slot so a single failure does not blank
// the rest of the page.
$queries = [
    'all' => [
        'title' => 'All Models (newest first)',
        'sql'   => "SELECT model_id, model_name, parameter_count, quantization, size_gb, release_date, is_installed,
use_case
                FROM " . DB_TABLE . "
                ORDER BY release_date DESC, model_name",
        'rows'  => [],
        'error' => null,
    ],
    'installed' => [
        'title' => 'Locally Installed Models, Largest First',
        'sql'   => "SELECT model_name, parameter_count, size_gb, use_case
                FROM " . DB_TABLE . "
                WHERE is_installed = 1
                ORDER BY parameter_count DESC",
        'rows'  => [],
        'error' => null,
    ],
    'summary' => [
```

```
      'title' => 'Fleet Summary (aggregates)',
      'sql'   => "SELECT
                    COUNT(*)                    AS total_models,
                    SUM(is_installed)           AS installed_count,
                    ROUND(SUM(size_gb), 2)      AS total_size_gb,
                    ROUND(AVG(parameter_count), 2)  AS avg_param_billions,
                    MAX(parameter_count)        AS largest_param_count,
                    MIN(release_date)           AS oldest_release,
                    MAX(release_date)           AS newest_release
                FROM " . DB_TABLE,
      'rows'  => [],
      'error' => null,
   ],
   'by_quant' => [
      'title' => 'Models Grouped by Quantization',
      'sql'   => "SELECT
                    quantization,
                    COUNT(*)                AS model_count,
                    ROUND(AVG(size_gb), 2)  AS avg_size_gb
                FROM " . DB_TABLE . "
                GROUP BY quantization
                ORDER BY model_count DESC, quantization",
      'rows'  => [],
      'error' => null,
   ],
];

if ($mysqli !== null) {
   foreach ($queries as $key => $query) {
      $err = null;
      $queries[$key]['rows']  = runSelect($mysqli, $query['sql'], $err);
      $queries[$key]['error'] = $err;
   }
   $mysqli->close();
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 9.2 — Query Table';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');

/**
 * Echoes a value or an em-dash when null/empty for cleaner table cells.
 *
 * @param  mixed $value Raw cell value from a result row.
 * @return string       HTML-safe display string.
 */
function cell(mixed $value): string
{
   if ($value === null || $value === '') {
      return '<span class="empty">&mdash;</span>';
```

```
    }
    return htmlspecialchars((string)$value, ENT_QUOTES, 'UTF-8');
}
?>
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Query Table — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
    <link rel="stylesheet" href="../../shared.css">
</head>

<body>
    <h1>Query Table</h1>

    <div class="header-info">
        <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
        <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
        <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
    </div>

<?php if ($connectError !== null): ?>
    <div class="error-summary">
        <h2>Connection Failed</h2>
        <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code> database.</p>
        <ul class="error-list">
            <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
        </ul>
        <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
    </div>

<?php else: ?>

    <h2>1. <?php echo htmlspecialchars($queries['all']['title'], ENT_QUOTES, 'UTF-8'); ?></h2>
    <div class="query-note">
        <strong>SQL:</strong>
        <code>SELECT * FROM <?php echo DB_TABLE; ?> ORDER BY release_date DESC, model_name</code>
    </div>
<?php if ($queries['all']['error'] !== null): ?>
    <p class="error-message">Query failed: <?php echo htmlspecialchars($queries['all']['error'], ENT_QUOTES, 'UTF-
8'); ?></p>
<?php else: ?>
    <table>
        <caption>All <?php echo count($queries['all']['rows']); ?> models</caption>
        <thead>
            <tr>
                <th>ID</th>
                <th>Model</th>
                <th>Params (B)</th>
```

```
            <th>Quant</th>
            <th>Size (GB)</th>
            <th>Released</th>
            <th>Installed</th>
            <th>Use Case</th>
          </tr>
        </thead>
        <tbody>
<?php if (empty($queries['all']['rows'])): ?>
          <tr><td colspan="8" class="empty">No rows. Run BrittaneyPopulateTable.php first.</td></tr>
<?php else: foreach ($queries['all']['rows'] as $row): ?>
          <tr>
            <td><?php echo (int)$row['model_id']; ?></td>
            <td><code><?php echo cell($row['model_name']); ?></code></td>
            <td><?php echo cell($row['parameter_count']); ?></td>
            <td><?php echo cell($row['quantization']); ?></td>
            <td><?php echo cell($row['size_gb']); ?></td>
            <td><?php echo cell($row['release_date']); ?></td>
            <td>
<?php if ((int)$row['is_installed'] === 1): ?>
              <span class="flag-true">Yes</span>
<?php else: ?>
              <span class="flag-false">No</span>
<?php endif; ?>
            </td>
            <td><?php echo cell($row['use_case']); ?></td>
          </tr>
<?php endforeach; endif; ?>
        </tbody>
      </table>
<?php endif; ?>

      <h2>2. <?php echo htmlspecialchars($queries['installed']['title'], ENT_QUOTES, 'UTF-8'); ?></h2>
      <div class="query-note">
        <strong>SQL:</strong>
        <code>SELECT model_name, parameter_count, size_gb, use_case FROM <?php echo DB_TABLE; ?> WHERE
is_installed = 1 ORDER BY parameter_count DESC</code>
      </div>
<?php if ($queries['installed']['error'] !== null): ?>
      <p class="error-message">Query failed: <?php echo htmlspecialchars($queries['installed']['error'], ENT_QUOTES,
'UTF-8'); ?></p>
<?php else: ?>
      <table>
        <caption><?php echo count($queries['installed']['rows']); ?> installed models</caption>
        <thead>
          <tr>
            <th>Model</th>
            <th>Params (B)</th>
            <th>Size (GB)</th>
            <th>Use Case</th>
          </tr>
        </thead>
```

```
      <tbody>
<?php if (empty($queries['installed']['rows'])): ?>
        <tr><td colspan="4" class="empty">No installed models found.</td></tr>
<?php else: foreach ($queries['installed']['rows'] as $row): ?>
        <tr>
          <td><code><?php echo cell($row['model_name']); ?></code></td>
          <td><?php echo cell($row['parameter_count']); ?></td>
          <td><?php echo cell($row['size_gb']); ?></td>
          <td><?php echo cell($row['use_case']); ?></td>
        </tr>
<?php endforeach; endif; ?>
      </tbody>
    </table>
<?php endif; ?>

    <h2>3. <?php echo htmlspecialchars($queries['summary']['title'], ENT_QUOTES, 'UTF-8'); ?></h2>
    <div class="query-note">
      <strong>SQL:</strong>
      <code>SELECT COUNT(*), SUM(is_installed), ROUND(SUM(size_gb),2), ROUND(AVG(parameter_count),2),
MAX(parameter_count), MIN(release_date), MAX(release_date) FROM <?php echo DB_TABLE; ?></code>
    </div>
<?php if ($queries['summary']['error'] !== null): ?>
    <p class="error-message">Query failed: <?php echo htmlspecialchars($queries['summary']['error'],
ENT_QUOTES, 'UTF-8'); ?></p>
<?php elseif (empty($queries['summary']['rows'])): ?>
    <p class="empty">No rows in table to aggregate.</p>
<?php else: $summary = $queries['summary']['rows'][0]; ?>
    <table class="data-display">
      <caption>Aggregate Statistics</caption>
      <tbody>
        <tr><td>Total models cataloged</td><td><?php echo cell($summary['total_models']); ?></td></tr>
        <tr><td>Installed locally</td><td><?php echo cell($summary['installed_count']); ?></td></tr>
        <tr><td>Combined disk footprint</td><td><?php echo cell($summary['total_size_gb']); ?> GB</td></tr>
        <tr><td>Average parameter count</td><td><?php echo cell($summary['avg_param_billions']); ?>
B</td></tr>
        <tr><td>Largest model</td><td><?php echo cell($summary['largest_param_count']); ?> B
parameters</td></tr>
        <tr><td>Oldest release date</td><td><?php echo cell($summary['oldest_release']); ?></td></tr>
        <tr><td>Newest release date</td><td><?php echo cell($summary['newest_release']); ?></td></tr>
      </tbody>
    </table>
<?php endif; ?>

    <h2>4. <?php echo htmlspecialchars($queries['by_quant']['title'], ENT_QUOTES, 'UTF-8'); ?></h2>
    <div class="query-note">
      <strong>SQL:</strong>
      <code>SELECT quantization, COUNT(*), ROUND(AVG(size_gb),2) FROM <?php echo DB_TABLE; ?> GROUP BY
quantization ORDER BY model_count DESC</code>
    </div>
<?php if ($queries['by_quant']['error'] !== null): ?>
    <p class="error-message">Query failed: <?php echo htmlspecialchars($queries['by_quant']['error'],
ENT_QUOTES, 'UTF-8'); ?></p>
```

```
<?php else: ?>
  <table>
    <caption>Counts per quantization scheme</caption>
    <thead>
      <tr>
        <th>Quantization</th>
        <th>Model Count</th>
        <th>Avg Size (GB)</th>
      </tr>
    </thead>
    <tbody>
<?php if (empty($queries['by_quant']['rows'])): ?>
      <tr><td colspan="3" class="empty">No rows to group.</td></tr>
<?php else: foreach ($queries['by_quant']['rows'] as $row): ?>
      <tr>
        <td><code><?php echo cell($row['quantization']); ?></code></td>
        <td><?php echo cell($row['model_count']); ?></td>
        <td><?php echo cell($row['avg_size_gb']); ?></td>
      </tr>
<?php endforeach; endif; ?>
    </tbody>
  </table>
<?php endif; ?>

<?php endif; /* connection ok */ ?>

  <div class="form-actions">
    <a href="BrittaneyIndex.php">Index</a>
    <a href="BrittaneyCreateTable.php" class="btn-secondary">Create Table</a>
    <a href="BrittaneyPopulateTable.php" class="btn-secondary">Populate Table</a>
    <a href="BrittaneyDropTable.php" class="btn-secondary">Drop Table</a>
  </div>
</body>

</html>
```

BrittaneyDropTable.php

```
<?php
/**
 * Module 9.2 — Drop Table (carried from Module 8.2)
 * CSD440 Server-Side Scripting
 *
 * Drops the bperrymorgan_ollama_models table from the course-supplied
 * baseball_01 database using MySQLi. Used between test runs to reset the
 * environment to a clean state. Uses DROP TABLE IF EXISTS so the script
 * is safe to run even when the table has already been removed.
 *
 * Reports whether the table existed before the DROP so the user can tell
 * the difference between a successful cleanup and an idempotent no-op.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-10
 * @php     >= 8.0
 */

// ── Configuration ──────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Checks whether a given table currently exists in the active database.
 *
 * Used before the DROP so the success message can distinguish "actually
```

```
 * removed something" from "no-op, table was already gone."
 *
 * @param  mysqli $mysqli Active MySQLi connection.
 * @param  string $table  Unqualified table name to check.
 * @return bool           True if the table exists, false otherwise.
 */
function tableExists(mysqli $mysqli, string $table): bool
{
    $escaped = $mysqli->real_escape_string($table);
    $result = $mysqli->query("SHOW TABLES LIKE '" . $escaped . "'");
    if (!($result instanceof mysqli_result)) {
        return false;
    }
    $exists = $result->num_rows > 0;
    $result->free();
    return $exists;
}

/**
 * Issues the DROP TABLE statement against the active connection.
 *
 * Uses DROP TABLE IF EXISTS so the script is safely idempotent — running
 * it on an already-dropped table does not error out.
 *
 * @param  mysqli      $mysqli Active MySQLi connection.
 * @param  string|null &$error Populated with the failure reason on error.
 * @return bool                True on success, false on error.
 */
function dropOllamaModelsTable(mysqli $mysqli, ?string &$error = null): bool
{
    if (!$mysqli->query("DROP TABLE IF EXISTS " . DB_TABLE)) {
        $error = $mysqli->error;
        return false;
    }
    return true;
}

// ── Execute ──────────────────────────────────────────────────────────
$connectError  = null;
$dropError     = null;
$existedBefore = false;
$dropped       = false;

$mysqli = connectToDatabase($connectError);
if ($mysqli !== null) {
    $existedBefore = tableExists($mysqli, DB_TABLE);
    $dropped       = dropOllamaModelsTable($mysqli, $dropError);
    $mysqli->close();
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
```

```
$assignmentTitle = 'Module 9.2 — Drop Table';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Drop Table — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
  <link rel="stylesheet" href="../../shared.css">
</head>

<body>
  <h1>Drop Table</h1>

  <div class="header-info">
    <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
    <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
    <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
  </div>

<?php if ($connectError !== null): ?>
  <div class="error-summary">
    <h2>Connection Failed</h2>
    <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code> database.</p>
    <ul class="error-list">
      <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
    </ul>
    <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
  </div>

<?php elseif (!$dropped): ?>
  <div class="error-summary">
    <h2>Drop Failed</h2>
    <p class="error-message">The <code><?php echo DB_TABLE; ?></code> table could not be dropped.</p>
    <ul class="error-list">
      <li><?php echo htmlspecialchars($dropError ?? 'Unknown error.', ENT_QUOTES, 'UTF-8'); ?></li>
    </ul>
  </div>

<?php else: ?>
  <div class="confirmation">
    <h2><?php echo $existedBefore ? 'Table Dropped' : 'Nothing To Drop'; ?></h2>

<?php if ($existedBefore): ?>
    <p class="success-message">
      The <code><?php echo DB_TABLE; ?></code> table was found and removed
      from the <code><?php echo DB_NAME; ?></code> database.
    </p>
```

```
<?php else: ?>
      <p class="success-message">
        The <code><?php echo DB_TABLE; ?></code> table did not exist, so the
        DROP statement completed as a no-op. Database is in a clean state.
      </p>
<?php endif; ?>

      <div class="query-note">
        <strong>SQL executed:</strong>
        <code>DROP TABLE IF EXISTS <?php echo DB_TABLE; ?></code>
      </div>

      <p>Next step: rebuild the table by running
        <code><a href="BrittaneyCreateTable.php">BrittaneyCreateTable.php</a></code>.</p>
    </div>
<?php endif; ?>

    <div class="form-actions">
      <a href="BrittaneyIndex.php">Index</a>
      <a href="BrittaneyCreateTable.php">Create Table</a>
      <a href="BrittaneyPopulateTable.php" class="btn-secondary">Populate Table</a>
      <a href="BrittaneyQueryTable.php" class="btn-secondary">Query Table</a>
    </div>
</body>

</html>
```